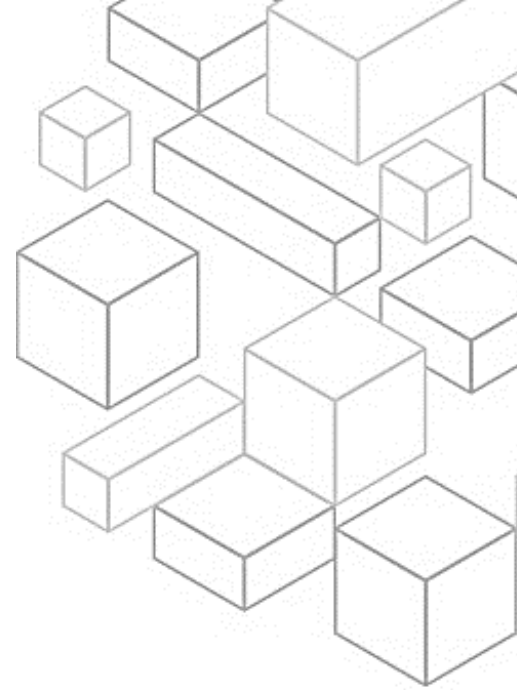




Module 4 Deep Dive on the Operational Excellence Pillar

1.1 AWS Well-Architected

Welcome to module four of AWS Well-Architected: Deep Dive on the Operational Excellence Pillar.

1.2 Learning objectives

In this module, you will get an overview of the operational excellence pillar of the AWS Well-Architected Framework. You will also learn the design principles and best practices of the operational excellence pillar.

1.3 Operational Excellence Pillar Overview

To begin, you will get an overview of the operational excellence pillar.

1.4 Pillars of Well-Architected

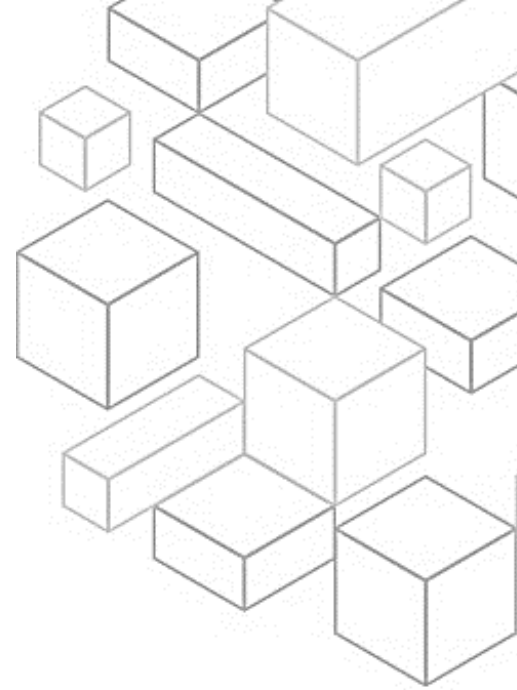
There are currently six pillars in the Well-Architected Framework: operational excellence, security, reliability, performance efficiency, cost optimization, and sustainability. These pillars are the foundations of the architecture for your technology solutions in the cloud. This module will focus on the operational excellence pillar.

1.5 What is the operational excellence pillar?

Some of the fundamentals of operational excellence are ensuring your workloads, processes, and procedures deliver business value to your organization. Your workloads need to deliver effective business value, and all the supporting functions around that workload need to reinforce that.



Operational excellence is about running your workloads effectively. Having a workload that is secure, cost optimized, reliable, and performant is fantastic, but if your teams cannot run and operate it effectively, then it becomes an overhead to the business.



1.6 Operational Excellence

Now that you understand what the operational excellence pillar is, you will dive deeper into the design principles of the operational excellence pillar.

1.7 Operational excellence

There are five design principles for operational excellence in the cloud. First is to perform operations as code. In the cloud, you can apply the same engineering discipline that you use for application code to your entire environment. You can define your entire workload, such as applications, infrastructure, and so on, as code and update it with code. You can script your operations procedures and automate their launch by invoking them in response to events. This can limit human error and drive consistent responses to events.

The second principle is to make frequent, small, and reversible changes. Design workloads so that you can update components regularly to increase the flow of beneficial changes into your workload. Make changes in small increments that you can reverse if they fail to help identify and resolve issues introduced to your environment, without affecting customers when possible.

The third principle is to refine operations procedures frequently. As you use operations procedures, look for opportunities to improve them. As you evolve your workload, evolve your procedures appropriately. Set up regular game days



to review and validate that all procedures are effective and that teams are familiar with them.

The fourth principle is to anticipate failure. Perform pre-mortem exercises to identify potential sources of failure so you can remove or mitigate them. Test your failure scenarios and validate your understanding of their impact. Test your response procedures to ensure they are effective and that teams are familiar with how to launch them. Set up regular game days to test workload and team responses to simulated events.

The last design principle is to learn from all operational failures. Drive improvement through lessons learned from all operational events and failures. Share what you learn across teams and through the entire organization.

1.8 Operational Excellence Best Practices

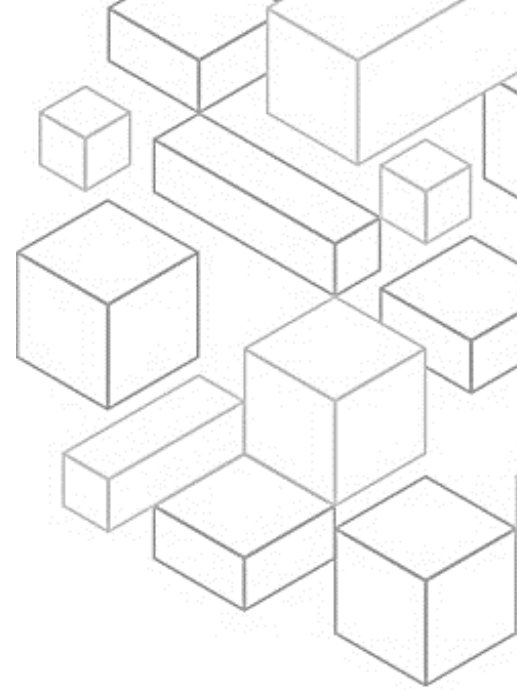
Now that you understand the operational excellence design principles, you will learn about the operational excellence best practices.

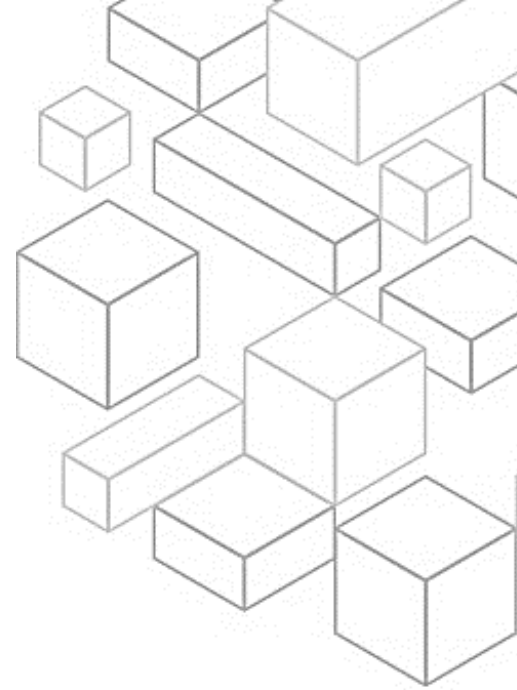
1.9 Operational excellence

To help you navigate the operational excellence best practices, the pillar looks at four different focus areas.

The first focus area is organization. You need to understand your organization's priorities, your organizational structure, and how your organization supports your team members, so that they can support your business outcomes.

Another focus area is to prepare. To prepare for operational excellence, you have to understand your workloads and their expected behaviors. You can then





design them to provide insight to their status and build the procedures to support them.

The third focus area is operate. Success is the achievement of business outcomes as measured by the metrics you define. By understanding the health of your workload and operations, you can identify when organizational and business outcomes may become at risk, or are at risk, and respond appropriately.

The last best practice focus area is evolve. Evolution is the continuous cycle of improvement over time. Implement frequent, small incremental changes based on the lessons learned from your operations activities and evaluate their success at bringing about improvement.

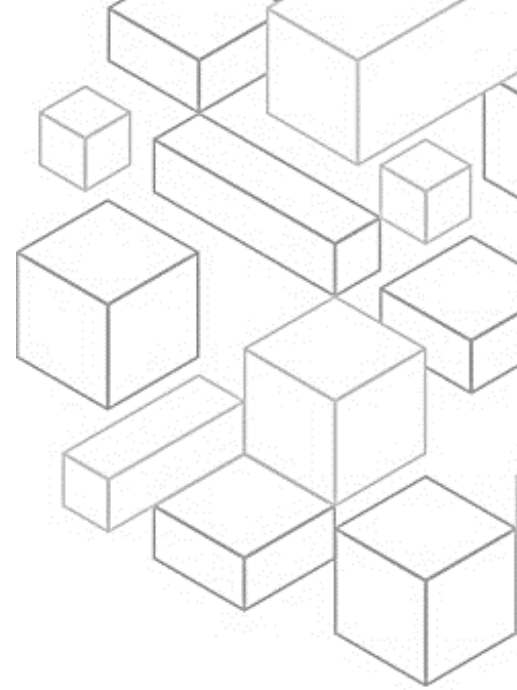
1.10 Organization

Organization is the first operational excellence best practice area.

1.11 Organization priorities

Your teams need to understand your entire workload, their role in it, and shared business goals to set the priorities that will drive business success. Well-defined priorities will maximize the benefits of your efforts. Review your priorities regularly so you can update them as your organization's needs change. You can consider a few best practices for this.

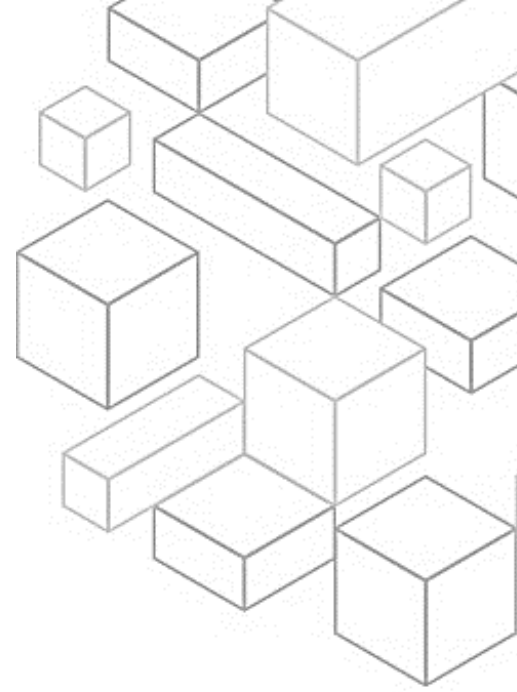
Evaluate external customer needs. Involve key stakeholders, including business, development, and operations teams, to determine where to focus efforts on external customer needs. This will ensure that you have a thorough understanding of the operations support required to achieve your desired business outcomes.



You should also evaluate internal customer needs. Involve key stakeholders when determining where to focus efforts, on internal customer needs, to ensure you understand the operations support required to achieve business outcomes. Use your established priorities to focus your improvement efforts where they will have the greatest impact. This might mean, for example, developing team skills, improving workload performance, reducing costs, automating runbooks, or enhancing monitoring. Update your priorities as needs change.

You should also consider evaluating governance requirements. Governance is the set of policies, rules, or frameworks that a company uses to achieve its business goals. Governance requirements are generated from within your organization. They can affect the types of technologies you choose or influence the way you operate your workload. Incorporate organizational governance requirements into your workload. Conformance is the ability to demonstrate that you have implemented governance requirements.

Evaluate compliance requirements. Regulatory, industry, and internal compliance requirements are an important driver for defining your organization's priorities. Your compliance framework may preclude you from using specific technologies or geographic locations. Apply due diligence if no external compliance frameworks are identified. Generate audits or reports that validate compliance. If you advertise that your product meets specific compliance standards, you must have an internal process for ensuring continuous compliance. Examples of compliance standards include PCI DSS, FedRAMP, and HIPAA. Applicable compliance standards are determined by

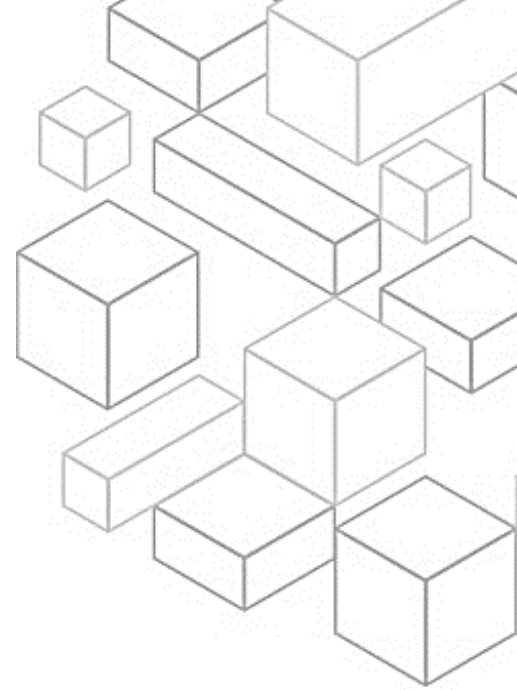


various factors, such as what types of data the solution stores or transmits and which geographic regions the solution supports.

You should also evaluate the threat landscape. Evaluate threats to the business such as competition, business risk and liabilities, operational risks, and information security threats. Maintain current information in a risk registry. Include the impact of risks when determining where to focus efforts.

Evaluate the impact of trade-offs between competing interests or alternative approaches, to help make informed decisions when determining where to focus efforts or choosing a course of action. For example, you may emphasize accelerating speed to market for new features over cost optimization. Or you may choose a relational database for non-relational data to simplify the effort to migrate a system, rather than migrating to a database optimized for your data type and updating your application.

The final best practice is to manage benefits and risks. This will help you to make informed decisions when determining where to focus your efforts. For example, it may be beneficial to deploy a workload with unresolved issues to make significant new features available to customers. It may be possible to mitigate associated risks, or it may become unacceptable for a risk to remain, in which case you will take action to address the risk. You might find that you want to emphasize a small subset of your priorities at some point in time. Use a balanced approach over the long term to ensure the development of needed capabilities and management of risk. Update your priorities as needs change.



1.12 Operating models

In this diagram, the vertical axis shows applications and platform. Applications refer to the workload serving a business outcome and can be custom developed or purchased software. Platform refers to the physical and virtual infrastructure and other software that supports that workload. On the horizontal axis, we have engineering and operations. Engineering refers to the development, building, and testing of applications and infrastructure.

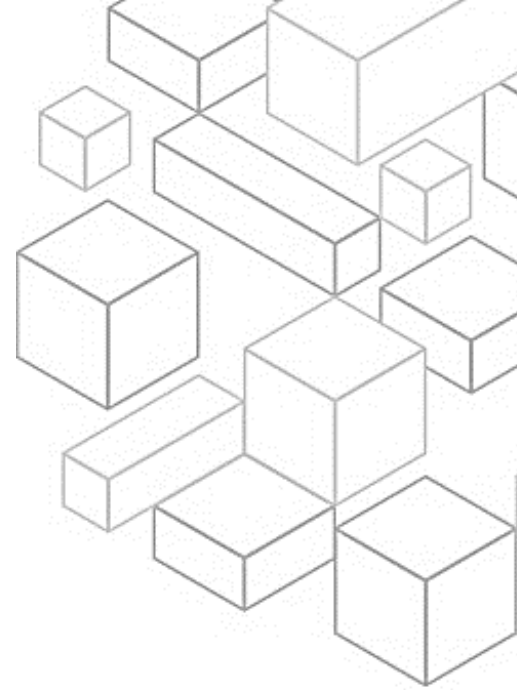
Operations is the deployment, update, and ongoing support of applications and infrastructure. There are other versions of this model that represent how these responsibilities tend to be distributed across teams. For more details, see the operational excellence pillar documentation.

1.13 Organizational culture

Provide support for your team members so that they can be more effective in taking action and supporting your business outcome. One way to do this is through executive sponsorship. Senior leadership clearly sets expectations for the organization and evaluates success. Senior leadership is the sponsor, advocate, and driver for the adoption of best practices and evolution of the organization.

Another way is by empowering team members. The workload owner has defined guidance and scope empowering team members to respond when outcomes are at risk.

Escalation mechanisms are used to get direction when events are outside of the defined scope. Team members also have mechanisms and are encouraged



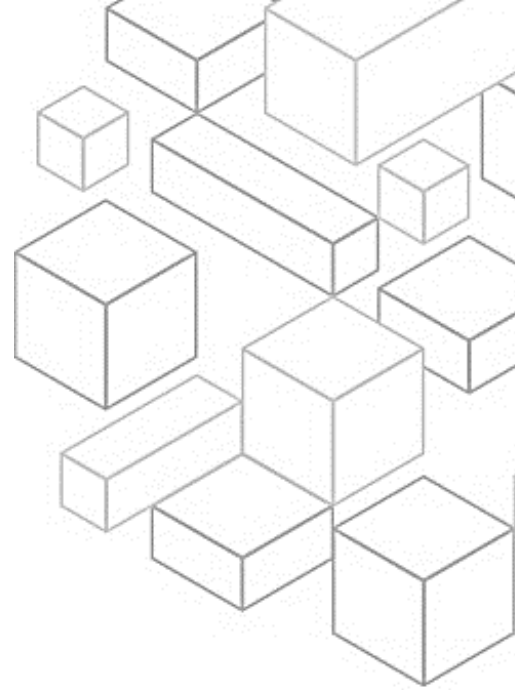
to escalate concerns to decision makers and stakeholders if they believe outcomes are at risk. Escalation should be performed early and often so that risks can be identified and prevented from causing incidents.

Communications should be timely, clear, and actionable. Mechanisms exist and are used to provide timely notice to team members of known risks and planned events.

Necessary context, details, and time are provided when possible to support determining if action is necessary, what action is required, and to take action in a timely manner. For example, providing notice of software vulnerabilities to expedite patching, or providing notice of planned sales promotions so that a change freeze can be implemented to avoid the risk of service disruption. Planned events can be recorded in a change calendar or maintenance schedule so that team members can identify pending activities.

Experimentation is encouraged as it can be a catalyst for turning new ideas into products and features. It accelerates learning and keeps team members interested and engaged. Team members are encouraged to experiment often to drive innovation. Even when an undesired result occurs, there is value in knowing what not to do. Team members are not punished for successful experiments with undesired results. Team members are empowered and encouraged to maintain and grow their skill sets.

Teams must grow their skill sets to adopt new technologies, and to support changes in demand and responsibilities in support of your workloads. Growth of skills in new technologies is frequently a source of team member satisfaction and supports innovation. Support your team members' pursuit and maintenance of industry certifications that validate and acknowledge their



growing skills. Cross train to promote knowledge transfer and reduce the risk of significant impact when you lose skilled and experienced team members with institutional knowledge. Provide dedicated structured time for learning.

Resource teams appropriately. Maintain team member capacity, and provide tools and resources to support your workload needs. Overtasking team members increases the risk of incidents resulting from human error. Investments in tools and resources, for example, providing automation for frequently performed activities, can scale the effectiveness of your team. This can make it possible for them to support additional activities.

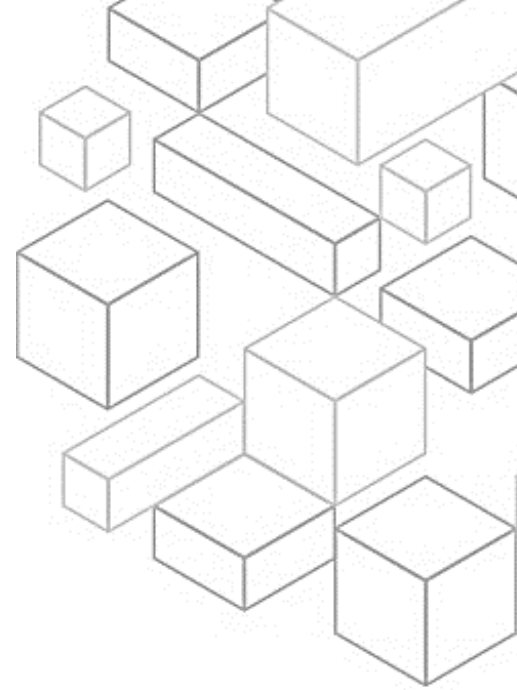
Diverse opinions are encouraged and sought within and across teams. Leverage cross-organizational diversity to seek multiple unique perspectives. Use this perspective to increase innovation, challenge your assumptions, and reduce the risk of confirmation bias. Grow inclusion, diversity, and accessibility within your teams to gain beneficial perspectives.

1.14 Prepare

To prepare for operational excellence, you have to understand your workloads and their expected behaviors. You can then design them to provide insight into their status and build procedures to support them.

1.15 Design Telemetry

Design your workload so that it provides the information necessary for you to understand its internal state. Examples include metrics, logs, events, and traces. This should be across all components in support of observability and investigating issues. Iterate to develop the telemetry necessary to monitor the

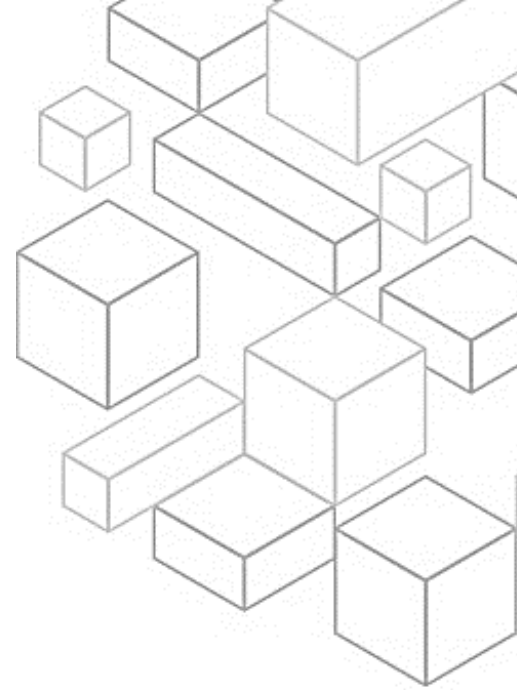


health of your workload, identify when outcomes are at risk, and drive effective responses.

Implement application telemetry, which is the foundation for observability of your workload. Your application should emit telemetry that provides insight into the state of the application and the achievement of business outcomes. From troubleshooting to measuring the impact of a new feature, application telemetry informs the way you build, operate, and evolve your workload. Application telemetry consists of metrics and logs. Metrics are diagnostic information, such as your pulse or temperature. Metrics are used collectively to describe the state of your application. Collecting metrics over time can be used to develop baselines and detect anomalies. Logs are messages that the application sends about its internal state or events that occur. Examples of events that are logged include error codes, transaction identifiers, and user actions.

Implement and configure workload telemetry. Design and configure your workload to emit information about its internal state and current status, for example, API call volume, HTTP status codes, and scaling events. Use this information to help determine when a response is required.

Implement user activity telemetry. Instrument your application code to emit information about user activity. Examples of user activity include click streams or started, abandoned, and completed transactions. Use this information to help understand how the application is used, patterns of usage, and to determine when a response is required. By capturing real user activity, you can build synthetic activity you can use to monitor and test your workload in production.



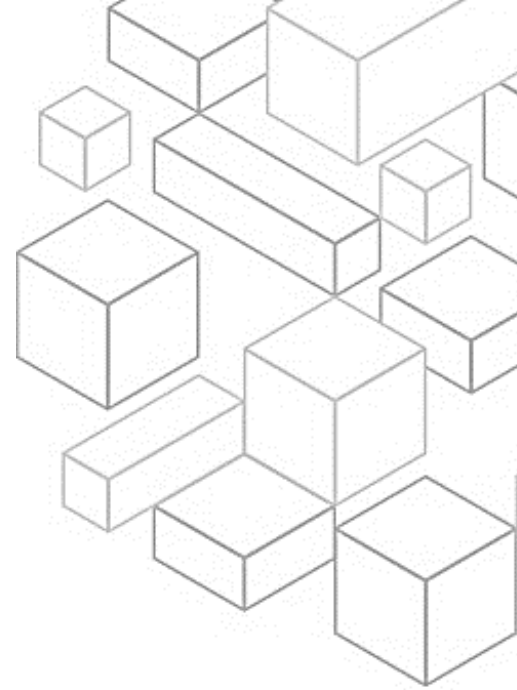
Implement dependency telemetry. Design and configure your workload to emit information about the status of resources it depends on. These are resources that are external to your workload. Examples of external dependencies include external databases, DNS, and network connectivity. Use this information to determine when a response is required and provide additional context on workload state.

Lastly, implement transaction traceability. Implement your application code and configure your workload components to emit events, which are initiated as a result of single logical operations and consolidated across various boundaries of your workload. Generate maps to see how traces flow across your workload and services. Gain insight into the relationships between components, and identify and analyze issues. Then, use the collected information to determine when a response is required and to assist you in identifying the factors contributing to an issue.

1.16 Design for operations

Adopt approaches that improve the flow of changes into production and that make refactoring, fast feedback on quality, and bug fixing possible. These accelerate beneficial changes entering production, limit issues deployed, and drive rapid identification and remediation of issues introduced through deployment activities. With AWS, you can view your entire workload, including applications, infrastructure, policy, governance, and operations, as code. It can all be defined and updated using code. This means you can apply the same engineering discipline that you use for application code to every element of your stack.

Use version control to initiate tracking of changes and releases.



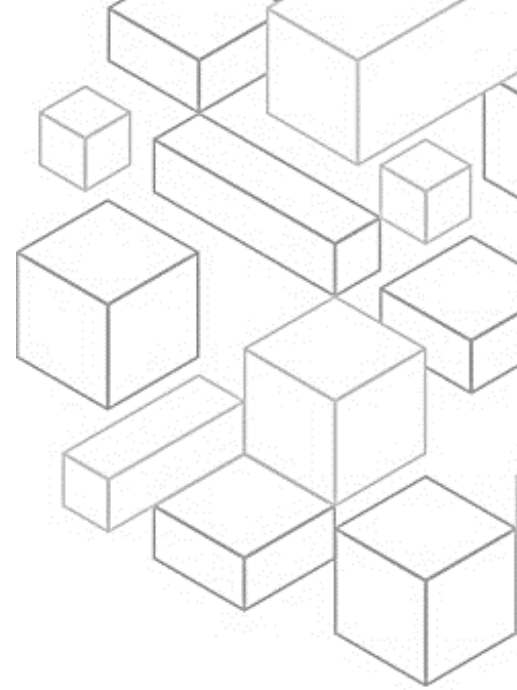
Also, test and validate changes. You need to test every change deployed to avoid errors in production. This best practice is focused on testing changes from version control to artifact build. Besides application code changes, testing should include infrastructure, configuration, security controls, and operations procedures. Testing takes many forms, from unit tests to software component analysis (SCA). Moving tests further to the left in the software integration and delivery process results in higher certainty of artifact quality. Your organization must develop testing standards for all software artifacts. Automated tests reduce toil and avoid manual test errors. Manual tests may be necessary in some cases. Developers must have access to automated test results to create feedback loops that improve software quality.

You can also use configuration management systems to make and track configuration changes. These systems reduce errors caused by manual processes and reduce the level of effort to deploy changes.

Use build and deployment management systems, which reduce errors caused by manual processes and reduce the level of effort to deploy changes. You can perform patch management to gain features, address issues, and remain compliant with governance.

Automate patch management to reduce errors caused by manual processes, and reduce the level of effort to patch. Patch and vulnerability management are part of your benefit and risk management activities. It is preferable to have immutable infrastructures and deploy workloads in verified known good states. Where that is not viable, you can patch in place.

Share design standards and best practices across teams to increase awareness and maximize the benefits of development efforts. Document them and keep



them up to date as your architecture evolves. If shared standards are enforced in your organization, it's critical that mechanisms exist to request additions, changes, and exceptions to standards. Without this option, standards become a constraint on innovation.

You can also implement practices to improve code quality and minimize defects. Some examples include test-driven development, code reviews, standards adoption, and pair programming. Incorporate these practices into your continuous integration and delivery process.

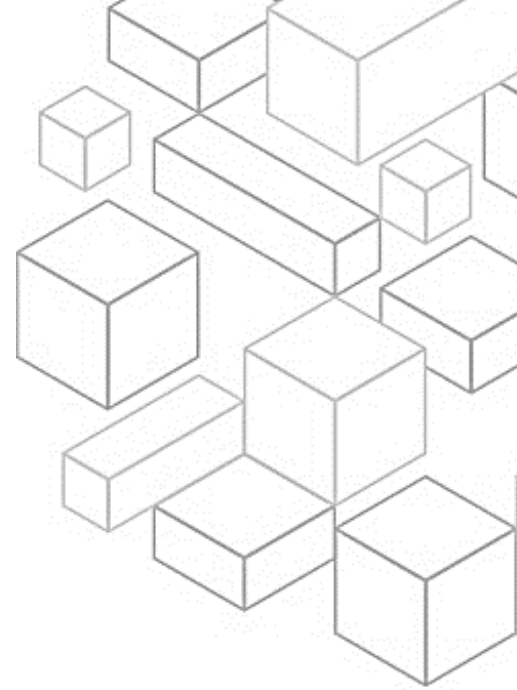
Additionally, use multiple environments to experiment, develop, and test your workload. Use increasing levels of controls as environments approach production to gain confidence your workload will operate as intended when deployed.

Making frequent, small, and reversible changes can reduce the scope and impact of a change. This eases troubleshooting, makes it possible to remediate faster, and provides the option to roll back a change.

Lastly, you should fully automate integration and deployment. Automate build, deployment, and testing of the workload. This reduces errors caused by manual processes and reduces the effort to deploy changes.

1.17 Mitigate deployment risks

There are also ways to mitigate deployment risks. First, plan for unsuccessful changes. Plan to revert to a known good state, or remediate in the production environment if a change does not have the desired outcome. This preparation reduces recovery time through faster responses.



Also, test changes and validate the results at all lifecycle stages to confirm new features and minimize the risk and impact of failed deployments.

Use deployment management systems to track and implement change. This reduces errors caused by manual processes and reduces the effort to deploy changes.

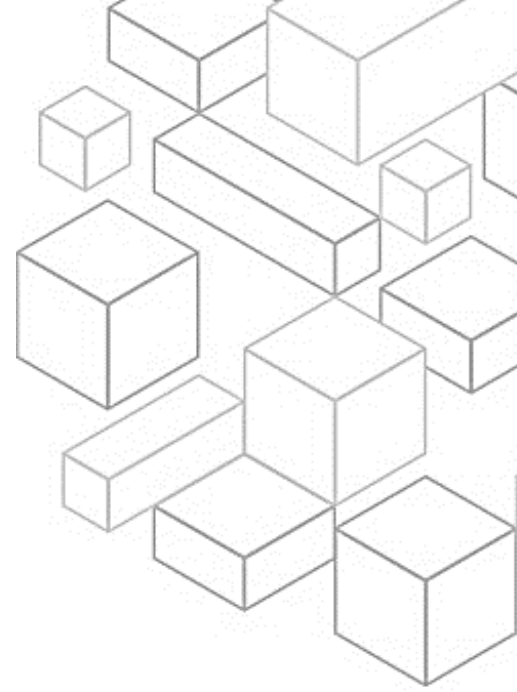
You can also test using limited deployments alongside existing systems to confirm desired outcomes prior to full scale deployment. For example, use deployment canary testing or one-box deployments.

Deploy using parallel environments. Implement changes onto parallel environments, and then transition over to the new environment. Maintain the prior environment until there is confirmation of successful deployment. Doing so minimizes recovery time by making it possible to roll back to the previous environment.

Deploy frequent, small, reversible changes to reduce the scope of a change. This results in easier troubleshooting and faster remediation with the option to roll back a change.

You can also fully automate integration and deployment. Automate build, deployment, and testing of the workload. This reduces errors caused by manual processes and reduces the effort to deploy changes.

Lastly, automate testing and rollback. Automate testing of deployed environments to confirm desired outcomes. Automate rollback to a previous known good state when outcomes are not achieved to minimize recovery time and reduce errors caused by manual processes.

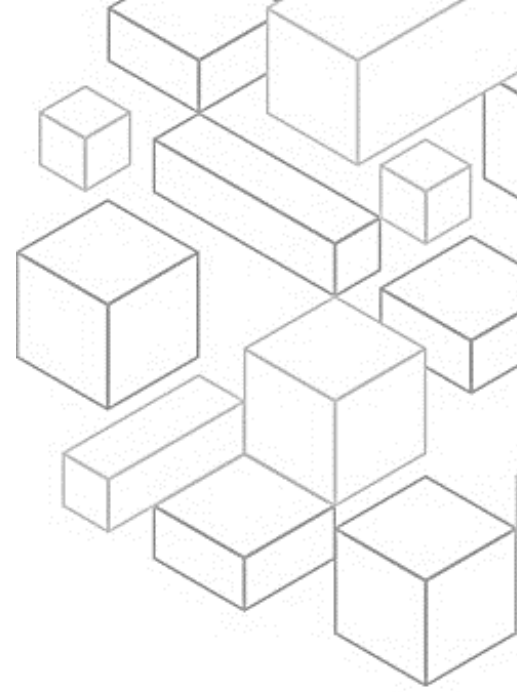


1.18 Operational readiness and change management

Evaluate the operational readiness of your workload, processes, procedures, and personnel to understand the operational risks related to your workload. Manage the flow of change into your environments. You should use a consistent process, including manual or automated checklists, to know when you are ready to go live with your workload or a change. This will also help you find any areas that you need to make plans to address. You will have runbooks that document your routine activities and playbooks that guide your processes for issue resolution. Use a mechanism to manage changes that supports the delivery of business value and help mitigate risks associated to change.

Ensure personnel capability by having a mechanism to validate that you have the appropriate number of trained personnel to support the workload. They must be trained on the platform and services that make up your workload. Provide them with the knowledge necessary to operate the workload. There must be enough personnel to support the normal operation of the workload and troubleshoot any incidents that occur. Have enough personnel so that you can rotate during on-call and vacations to avoid burnout.

You should also ensure a consistent review of operational readiness. Use Operational Readiness Reviews, or ORRs, to validate that you can operate your workload. ORR is a mechanism developed at Amazon to validate that teams can safely operate their workloads. It is a review and inspection process using a checklist of requirements and a self-service experience that teams use to certify their workloads. ORRs include best practices from lessons learned from our years of building software. The checklist includes architectural

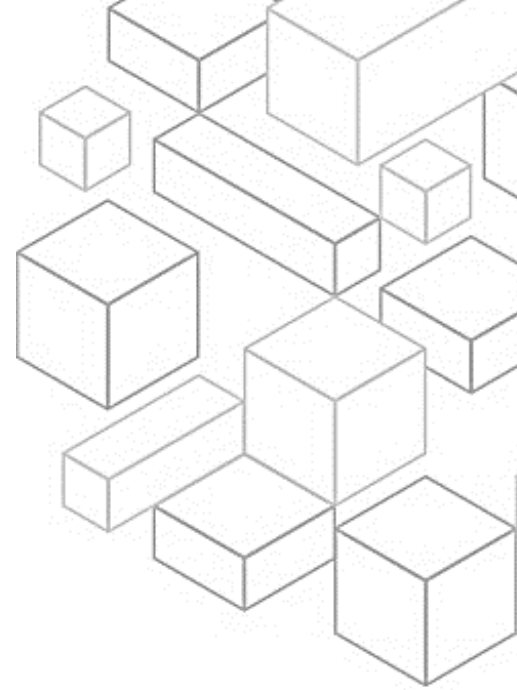


recommendations, operational processes, event management, and release quality. Our Correction of Error process is a major driver of these items. Your own post-incident analysis should drive the evolution of your own ORR. An ORR is not only about following best practices, but preventing the recurrence of events that you've seen before. Security, governance, and compliance requirements can also be included in an ORR.

Use runbooks, which are documented processes to achieve specific outcomes, to perform procedures. Runbooks consist of a series of steps that someone follows to get something done. Runbooks have been used in operations going back to the early days of aviation. In cloud operations, we use runbooks to reduce risk and achieve desired outcomes. At its simplest, a runbook is a checklist to complete a task.

Use playbooks, which are step-by-step guides used to investigate an incident. When incidents happen, you can use playbooks to investigate, scope impact, and identify a root cause. You can use playbooks for a variety of scenarios, from failed deployments to security incidents. In many cases, playbooks identify the root cause that a runbook is used to mitigate. Playbooks are an essential component of your organization's incident response plans.

You should also make informed decisions to deploy systems and changes. Have processes in place for successful and unsuccessful changes to your workload. A pre-mortem is an exercise where a team simulates a failure to develop mitigation strategies. Use pre-mortems to anticipate failure and create procedures where appropriate. Evaluate the benefits and risks of deploying changes to your workload. Verify that all changes comply with governance.



Facilitate support plans for production workloads. Make sure to support any software and services that your production workload relies on. Select an appropriate support level to meet your production service-level needs. Support plans for these dependencies are necessary in case of service disruptions or software issues. Document support plans and how to request support for all service and software vendors. Implement mechanisms that verify that support points of contacts are kept up to date.

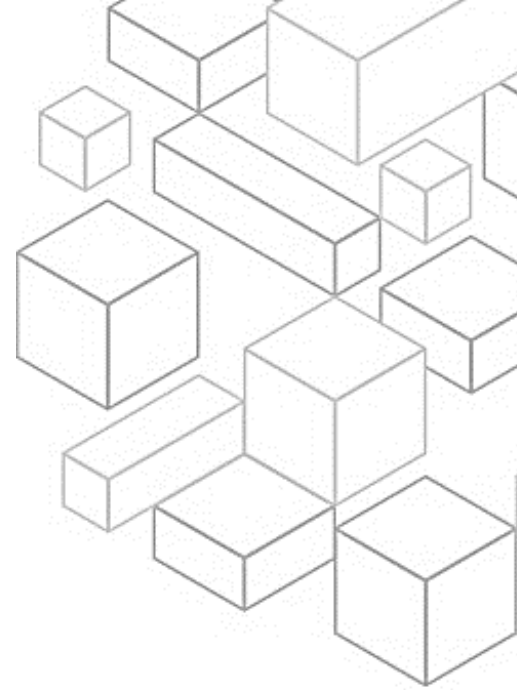
1.19 Operate

Success is the achievement of business outcomes as measured by the metrics you define. By understanding the health of your workload and operations, you can identify when organizational and business outcomes may become at risk, or are at risk, and respond appropriately.

1.20 Understanding workload health

Define, capture, and analyze workload metrics to gain visibility to workload events so that you can take appropriate action. Your team should be able to understand the health of your workload easily. You will want to use metrics based on workload outcomes to gain useful insights. You should use these metrics to implement dashboards with business and technical viewpoints that will help team members make informed decisions.

First, identify key performance indicators, or KPIs, based on desired business outcomes and customer outcomes. Desired business outcomes can include order rate, customer retention rate, and profit versus operating expense. Customer satisfaction is an example of customer outcomes. Evaluate KPIs to determine workload success.

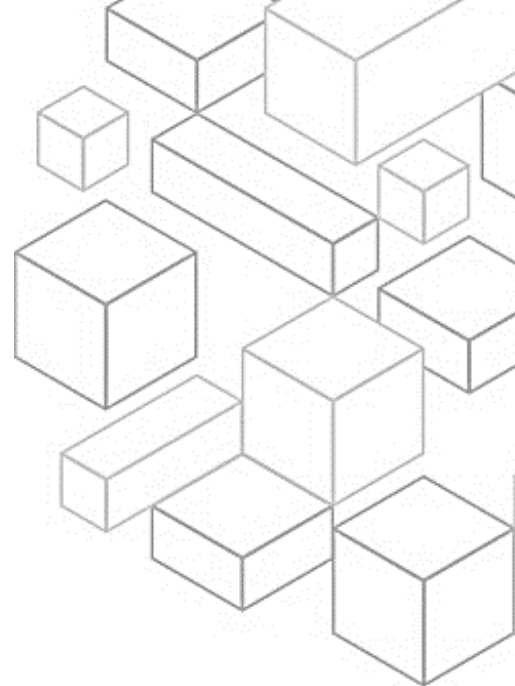


Also, define workload metrics, which measure the health of the workload. Workload health is measured by the achievement of business outcomes or KPIs and the state of workload components and applications. Examples of KPIs include abandoned shopping carts, orders placed, cost, price, and allocated workload expense. While you may collect telemetry from multiple components, select a subset that provides insight into the overall workload health. Adjust workload metrics over time as business needs change.

Collect and analyze workload metrics. Perform regular, proactive reviews of these metrics to identify trends and determine if a response is necessary and validate the achievement of business outcomes. Aggregate metrics from your workload applications and components to a central location. Use dashboards and analytics tools to analyze telemetry and determine workload health. Implement a mechanism to conduct workload health reviews on a periodic basis with stakeholders in your organization.

Also, establish workload metrics baselines to help you understand workload health and performance. Using baselines, you can identify under-performing and over-performing applications and components. A workload baseline adds to your ability to mitigate issues before they become incidents. Baselines are foundational in developing patterns of activity and implementing anomaly detection when metrics deviate from expected values.

Learn expected patterns of activity for workload. Establish patterns of workload activity to identify anomalous behavior so you can respond appropriately if required.



You should also raise an alert when workload outcomes are at risk so that you can respond appropriately if necessary. Ideally, you have previously identified a metric threshold that you can alarm upon or an event that you can use to invoke an automated response.

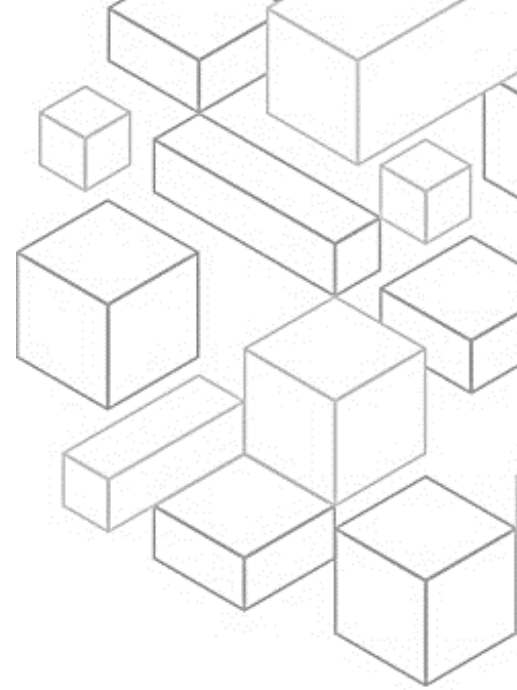
Also, raise an alert when workload anomalies are detected so that you can respond appropriately if necessary. Your analysis of your workload metrics over time may establish patterns of behavior that you can quantify sufficiently to define an event or raise an alarm in response.

Validate the achievement of outcomes and the effectiveness of KPIs and metrics. Create a business-level view of your workload operations to help you determine if you are satisfying needs and to identify areas that need improvement to reach business goals. Validate the effectiveness of KPIs and metrics and revise them if necessary.

1.21 Understanding operational health

Define, capture, and analyze operations metrics to gain visibility to workload events so that you can take appropriate action. Your team should be able to understand the health of your operations easily. You will want to use metrics based on operations outcomes to gain useful insights. You should use these metrics to implement dashboards with business and technical viewpoints that will help team members make informed decisions.

Identify KPIs based on desired business outcomes, such as new features delivered, and customer outcomes, such as customer support cases. Evaluate KPIs to determine operations success. Also, define operations metrics to



measure the achievement of KPIs, for example, successful deployments and failed deployments.

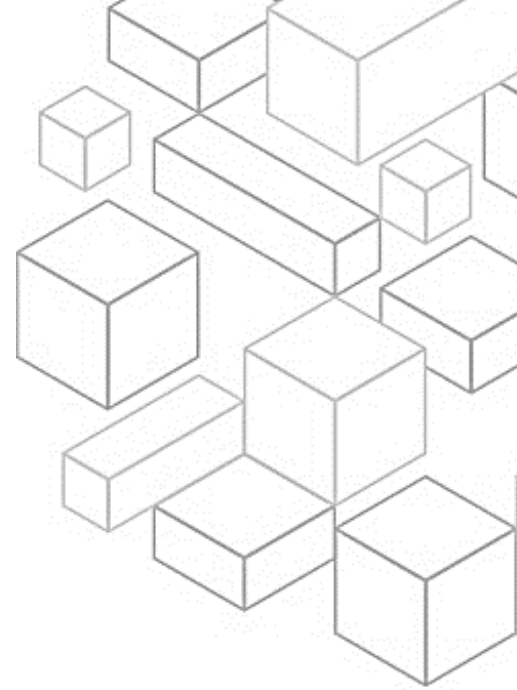
Define operations metrics to measure the health of operations activities such as mean time to detect an incident, or MTTD, and mean time to recovery, or MTTR, from an incident. Evaluate metrics to determine if operations are achieving desired outcomes, and to understand the health of your operations activities.

Collect and analyze operations metrics. Perform regular proactive reviews of metrics to identify trends and determine where appropriate responses are needed. You should aggregate log data from your operations activities and operations API calls, into a service such as Amazon CloudWatch Logs. Generate metrics from observations of necessary log content to gain insight into the performance of operations activities.

Establish baselines for metrics to provide expected values as the basis for comparison and identification of under- and over-performing operations activities.

You should also learn expected patterns of activity for operations. Establish patterns of operations activity to identify anomalous behavior so that you can respond appropriately if required.

Alert when operations outcomes are at risk. Whenever they are at risk, an alert must be raised and acted upon. Operations outcomes are any activity that supports a workload in production. This includes everything from deploying new versions of applications to recovering from an outage. Operations outcomes must be treated with the same importance as business outcomes. Software teams should identify key operations metrics and activities and build alerts for them. Alerts must be timely and actionable. If an alert is raised, a



reference to a corresponding runbook or playbook should be included. Alerts without a corresponding action can lead to alert fatigue.

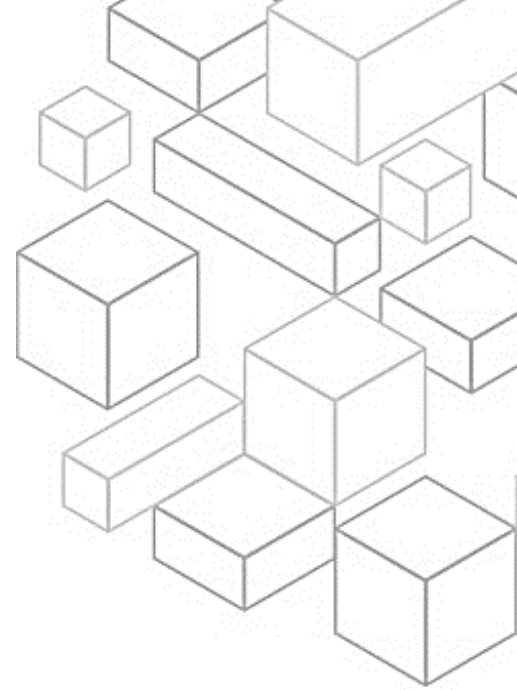
Alert when operations anomalies are detected. Raise an alert when operations anomalies are detected so that you can respond appropriately if necessary. Your analysis of your operations metrics over time may establish patterns of behavior that you can quantify sufficiently to define an event or raise an alarm in response.

Validate the achievement of outcomes and the effectiveness of KPIs and metrics. Create a business-level view of your operations activities to help you determine if you are satisfying needs and to identify areas that need improvement to reach business goals. Validate the effectiveness of KPIs and metrics and revise them if necessary.

1.22 Responding to events

You should anticipate operational events. These may include planned events such as sales promotions, deployments, and failure tests. They may also include unplanned events such as surges in utilization and component failures. You should use your existing runbooks and playbooks to deliver consistent results when you respond to alerts. A role or a team accountable for the response and escalations should own defined alerts. You will also want to know the business impact of your system components and use this to target efforts when needed. You should perform a root-cause analysis after events, and then prevent recurrence of failures or document workarounds.

Your organization should have processes to handle events, incidents, and problems. Events are things that occur in your workload but may not need



intervention. Incidents are events that require intervention. Problems are recurring events that require intervention or cannot be resolved. You need processes to mitigate the impact of these events on your business and make sure that you respond appropriately.

You should have a process per alert. Have a well-defined response (runbook or playbook), with a specifically identified owner, for any event for which you raise an alert.

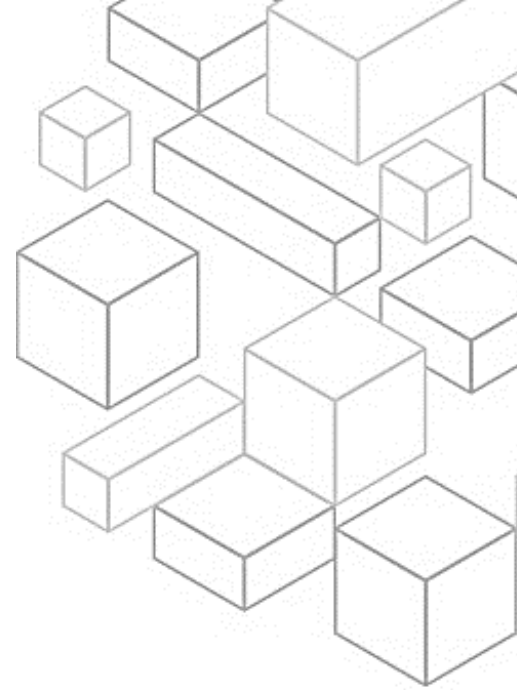
This ensures effective and prompt responses to operations events and prevents actionable events from being obscured by less valuable notifications.

You should also prioritize operational events based on business impact. Ensure that when multiple events require intervention, those that are most significant to the business are addressed first. Impacts can include loss of life or injury, financial loss, or damage to reputation or trust.

Define escalation paths in your runbooks and playbooks, including what sets off escalation, and procedures for escalation. Specifically identify owners for each action to ensure effective and prompt responses to operations events. Identify when a human decision is required before an action is taken. Work with decision makers to have that decision made in advance, and the action preapproved, to avoid long Mean Time to Repair, or MTTR, waiting for a response.

Define and test a communication plan for system outages that you can rely on to keep your customers and stakeholders informed during outages.

Communicate directly with your users both when the services they use are impacted and when services return to normal.



You can also communicate status through dashboards. Provide dashboards tailored to their target audiences, such as internal technical teams, leadership, and customers, to communicate the current operating status of the business and provide metrics of interest.

Lastly, automate responses to events to reduce errors caused by manual processes, and to ensure prompt and consistent responses.

1.23 Evolve

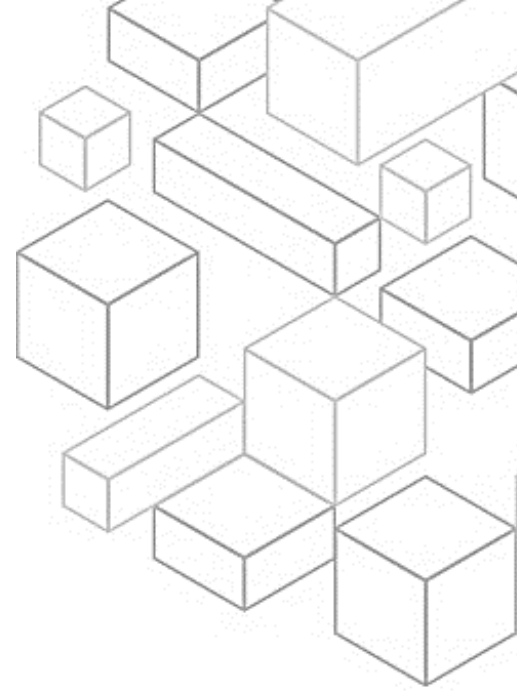
Evolution is the continuous cycle of improvement over time. Implement frequent small incremental changes based on the lessons learned from your operations activities and evaluate their success at bringing about improvement.

1.24 Learn, share, and improve

It's essential that you regularly provide time for analysis of operations activities, analysis of failures, experimentation, and making improvements. When things fail, you will want to ensure that your team, as well as your larger engineering community, learns from those failures. You should analyze failures to identify lessons learned and plan improvements. You will want to regularly review your lessons learned with other teams to validate your insights.

You should have a process for continuous improvement. Evaluate your workload against internal and external architecture best practices. Conduct workload reviews at least once per year. Prioritize improvement opportunities into your software development cadence.

You should also perform post-incident analysis. Review customer-impacting events and identify contributing factors and preventative actions. Use this



information to develop mitigations to limit or prevent recurrence. Develop procedures for prompt and effective responses. Communicate contributing factors and corrective actions as appropriate, tailored to target audiences.

Implement feedback loops, which can provide actionable insights that drive decision making. Build feedback loops into your procedures and workloads. This helps you identify issues and areas that need improvement. They also validate investments made in improvements. These feedback loops are the foundation for continuously improving your workload.

Knowledge management helps team members find the information to perform their job. In learning organizations, information is freely shared, which empowers individuals. The information can be discovered or searched. Information is accurate and up to date. Mechanisms exist to create new information, update existing information, and archive outdated information. The most common example of a knowledge management platform is a content management system like a Wiki.

Identify drivers for improvement to help you evaluate and prioritize opportunities.

You can also validate insights. Review your analysis results and responses with cross-functional teams and business owners. Use these reviews to establish common understanding, identify additional impacts, and determine courses of action. Adjust responses as appropriate.

Regularly perform retrospective analysis of operations metrics with cross-team participants from different areas of the business. Use these reviews to identify opportunities for improvement, potential courses of action, and to share



lessons learned. Look for opportunities to improve in all of your environments, such as development, test, and production.

You should also make sure to document and share lessons learned from the operations activities so that you can use them internally and across teams. Sharing what your teams learn can increase the benefit across your organization. You will want to share information and resources to prevent avoidable errors and ease development efforts. This will help you focus on delivering desired features.

Lastly, dedicate time and resources within your processes to make continuous incremental improvements possible.

1.27 Summary

In this module, you learned about the operational excellence pillar. This started with an overview and included in depth discussion on the value proposition, design principles, and best practices of the operational excellence pillar.

1.28 Thank you

Thank you for your participation!

